



Triggers

Ing. Ricardo Tillero

¿Qué son?

TRIGGERS

*“Un **trigger** o **disparador** es un objeto que se almacena en la base de datos asociado a una tabla y se activa cuando ocurre un evento en particular para esa tabla”*

- Los trigger contienen sentencias que se ejecutan cuando el trigger se activa.
- Los eventos que pueden activar un trigger son:
 - Se inserta una nueva fila en la tabla (INSERT).
 - Se borra una o mas filas de la tabla (DELETE).
 - Se actualiza una o mas filas de la tabla (UPDATE).
- También es posible definir en que momento se activará un trigger, esto es, si se activará antes (BEFORE) o después (AFTER) que se produzca el evento.
- En algunas versiones de MySQL, no se permite que haya dos triggers asociados a una misma tabla que correspondan al mismo momento y evento.

- Implementar reglas de consistencia de datos no provistas por el modelo relacional.
- Verificar y prevenir que se inserten datos inválidos.
- Replicación de datos.
- **Auditoria:** monitorear la actividad de los cambios sobre los datos.
- Acciones en cascadas.
- Autorización y seguridad.

Creación de triggers:

- Se utiliza la sentencia **CREATE TRIGGER**.

```
CREATE TRIGGER <nombre del trigger> {BEFORE|AFTER}  
      {INSERT|UPDATE|DELETE} ON <nombre de tabla> FOR EACH ROW;
```

Donde:

- **{BEFORE|AFTER}**: cuando se ejecuta: antes o después.
- **{INSERT|UPDATE|DELETE}**: que evento lo activa.
- **<nombre de tabla>**: tabla donde se produce el evento
- **<tipo>**: cualquier tipo válido de MySQL: int,char,date,etc.
- **<sentencia>**: Qué se ejecuta. Cualquier sentencia válida para un Stored Procedure, incluyendo: sentencias SQL, sentencias compuestas, llamadas a SP, condicionales, etc. Existen restricciones, por ejemplo: dentro de un trigger no es posible modificar la tabla que activó el trigger.

Creación de triggers - ejemplo:

- Crearemos un trigger que registre la fecha y hora de todos los cambios de saldo que se producen sobre las cuentas.
- Para almacenar estos datos creamos la siguiente tabla:

```
USE bank;
```

```
CREATE TABLE movimientos(  
    numero INT UNSIGNED AUTO_INCREMENT NOT NULL,  
    cuenta INT UNSIGNED NOT NULL,  
    fecha DATE NOT NULL,  
    hora TIME NOT NULL,  
    saldo_anterior DECIMAL(7,2) NOT NULL,  
    saldo_posterior DECIMAL(7,2) NOT NULL,  
    CONSTRAINT pk_movimientos  
    PRIMARY KEY(numero),  
    CONSTRAINT fk_movimientos_cuentas  
    FOREIGN KEY(cuenta) REFERENCES cuentas(numero)  
)ENGINE=InnoDB;
```

Creación de triggers - ejemplo:

```
delimiter $$ # define "$$" como delimitador
```

```
CREATE TRIGGER cuentas_update AFTER UPDATE ON cuentas FOR EACH ROW  
BEGIN
```

```
    INSERT INTO movimientos(cuenta, fecha, hora, saldo_anterior, saldo_posterior)  
    VALUES(OLD.numero, curdate(), curtime(), OLD.saldo, NEW.saldo);
```

```
END; $$
```

```
Delimiter ; # reestablece el ";" como delimitador
```

- Dado que “;” se usa para separar las sentencias dentro del trigger y también se usa para separar las sentencias SQL, por eso se cambia el delimitador a “&&”, “//” o “\$\$” para saber donde termina la sentencia **CREATE TRIGGER**. Atención: no dejar espacios después del delimitador o del “;”.
- Las **variables especiales de transición OLD y NEW** hacen referencia a los valores de la fila afectada **antes y después** de la modificación respectivamente.

Creación de triggers - ejemplo:

- **NEW** contiene los valores de la fila afectada después de ser modificada.
- Si el evento es **INSERT** solo se puede utilizar **NEW** y si es **DELETE** solo se puede usar **OLD**. Con **UPDATE** se pueden usar ambas.
- Comprobamos el funcionamiento del trigger:

```
mysql> SELECT * FROM movimientos;  
Empty set (0.00 sec)
```

```
mysql> UPDATE cuentas set saldo=5000 WHERE numero<=2;  
Query OK, 2 row affected (0.01 sec)  
Rows matched: 2 Changed: 2 Warnings: 0
```

```
mysql> SELECT * FROM movimientos;
```

numero	cuenta	fecha	hora	saldo_anterior	saldo_posterior
1	1	<actual>	<actual>	1000.00	5000.00
2	2	<actual>	<actual>	2000.00	5000.00

```
2 rows in set (0.00 sec)
```